

Chapter 9: Exceptions

EEC 3150

Trevecca Nazarene University

Outline

1 Handling Exceptions

2 Raising Exceptions

Handling Exceptions

Handling Exceptions

Defn: exception

Exceptions are errors in a Python script detected by the interpreter

They come in two main classes:

- *Syntax errors* occur when you write code that violates the rules of Python's syntax; detected by the interpreter before execution
 - Missing a parenthesis or quote
 - Misspelled keyword
 - Using a variable before it's defined
- *Runtime errors* occur during the execution of a script; cannot be detected by the interpreter before execution
 - Dividing by zero
 - Out of bounds indexing of a list
 - Trying to read a file that doesn't exist

Unless handled properly, exceptions always halt your program's execution.
How can we avoid this?

Handling Exceptions

Defn: exception handling

Exception handling (or error handling) is the process of detecting and dealing with errors in programs

Defn: the try except block

- We can use a try except block to perform error handling in Python
- This will give us a plan B in case our code throws an error
- It's similar to an if else block

```
try:  
    code that might throw an error  
except:  
    code to run in case of error
```

- The code in the try portion is run first
- If no error, then the block is finished
- If there is an error, then the except portion is run

First try except example:

Handling indexing a list out of bounds

```
try:  
    y = x[10]  
except:  
    y = x[-1]
```

- Try to store the 10th element of x in y
- If it doesn't exist, use the last element instead
- Either way, y gets defined

Handling Exceptions

Reading floats from `input()`:

- Read a float using `input()` and store it in a variable
- If the input can't be cast, print a message and set the variable to `nan`

Reading files that don't exist:

- Read some files and put their contents in a list
- If the files don't exist, append an empty string to the list
- Print out a message saying whether each file was successfully read or not

Handling Exceptions

Handling multiple kinds of errors:

- There are many types of exceptions (<https://docs.python.org/3/library/exceptions.html>)
- We can handle multiple types of exceptions by having multiple `except` blocks
- NOTE: all of the below error types aren't real, they're just for illustrative purposes

```
try:  
    code A  
except ErrorType1:  
    code B  
except ErrorType2, ErrorType3:  
    code C  
except:  
    code D
```

- If `code A` raises a type 1 error, run `code B`
- If `code A` raises either type 2 or 3 error, run `code C`
- If `code A` raises any other type of error, run `code D`

Handling Exceptions

Handling multiple kinds of errors:

```
def get_ratios(x, y):  
  
    ratios = []  
  
    for i in range(len(x)):  
        try:  
            ratios.append(x[i]/y[i])  
        except ZeroDivisionError:  
            ratios.append(float('nan'))  
        except TypeError:  
            ratios.append(float('nan'))  
  
    return ratios
```

Handling Exceptions

The entire block: try except else finally

There is an expanded try except block that has even more blocks which run under different conditions

```
try:
    try to run this code
except:
    run this if there were errors
else:
    run this if there weren't errors
finally:
    run this in either case
```

Raising Exceptions

Raising Exceptions

Manually raising an exception with `raise`:

- It is possible to manually raise an exception with the `raise` keyword:
- Syntax: `raise ErrorType(message)`
- Ex: add to lists, raise exception if they aren't the same length

```
def add(x, y):  
    if len(x) != len(y):  
        raise ValueError('arguments of add() must be same length')  
  
    sums = []  
  
    for i,j in zip(x,y):  
        sums.append(i+j)  
  
    return sums
```

Raising Exceptions

Checking conditions with assert:

- The `assert` keyword will raise an exception if a specified condition isn't met (useful for debugging)
- Syntax: `assert condition, message`
- Ex: checking list were copied properly

```
x = [1,2,3]
y = x
# y = x.copy()
```

```
assert id(x) != id(y)
```

- The first way of copying will assign `x, y` to the same location in memory (equal ids)
- Therefore, `assert` will raise an `AssertionError` to let you know that the condition IS NOT met
- The second way (with `x.copy()`) will not raise the exception